

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
По дисциплине «Проектирование и реализация баз данных»
Работа с БД в СУБД MongoDB

Выполнил: Востров Илья Анатольевич
Преподаватель: Говорова Марина Михайловна

Санкт-Петербург, 2026

Содержание

1	Лабораторная работа 6.1. Введение в СУБД MongoDB. Установка MongoDB	2
1.1	Цель работы	2
1.2	Выполнение	2
1.3	Контрольные вопросы (6.1)	4
2	Лабораторная работа 6.2. Работа с БД в СУБД MongoDB	6
2.1	Цель работы	6
2.2	Выполнение практических заданий	6
2.3	Контрольные вопросы (6.2)	21
2.4	Вывод	21

1 Лабораторная работа 6.1. Введение в СУБД MongoDB. Установка MongoDB

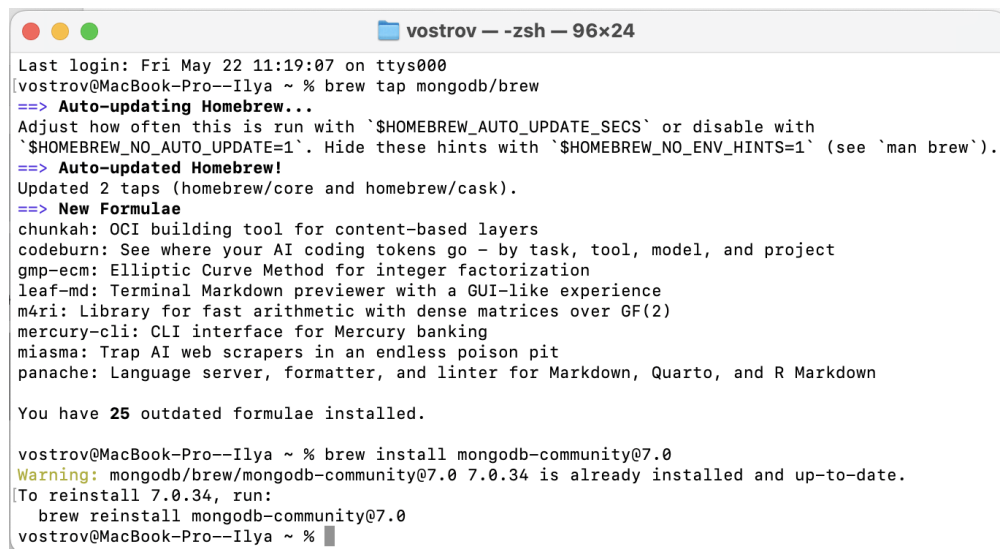
1.1 Цель работы

Овладеть практическими навыками установки СУБД MongoDB.

1.2 Выполнение

Часть 1-2: Установка и настройка путей

В связи с использованием ОС macOS, установка была произведена через пакетный менеджер Homebrew. Выполнена настройка путей и запуск сервиса.



```
vostrov -- zsh -- 96x24
Last login: Fri May 22 11:19:07 on ttys000
vostrov@MacBook-Pro--Ilya ~ % brew tap mongodb/brew
==> Auto-updating Homebrew...
Adjust how often this is run with `HOMEBREW_AUTO_UPDATE_SECS` or disable with
`HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
==> New Formulae
chunkah: OCI building tool for content-based layers
codeburn: See where your AI coding tokens go - by task, tool, model, and project
gmp-ecm: Elliptic Curve Method for integer factorization
leaf-md: Terminal Markdown previewer with a GUI-like experience
m4ri: Library for fast arithmetic with dense matrices over GF(2)
mercury-cli: CLI interface for Mercury banking
miasma: Trap AI web scrapers in an endless poison pit
panache: Language server, formatter, and linter for Markdown, Quarto, and R Markdown

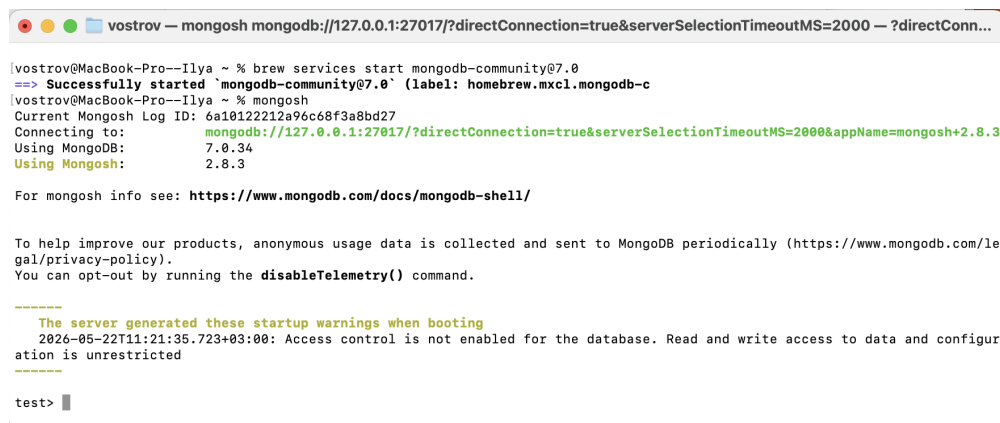
You have 25 outdated formulae installed.

vostrov@MacBook-Pro--Ilya ~ % brew install mongodb-community@7.0
Warning: mongodb/brew/mongodb-community@7.0 7.0.34 is already installed and up-to-date.
To reinstall 7.0.34, run:
  brew reinstall mongodb-community@7.0
vostrov@MacBook-Pro--Ilya ~ %
```

Рис. 1: Установка и настройка

Часть 3: Запуск сервера и клиента

Сервер запущен в фоновом режиме, осуществлен вход в интерактивную оболочку mongosh.



```
vostrov -- mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000 -- ?directConn...

vostrov@MacBook-Pro--Ilya ~ % brew services start mongodb-community@7.0
==> Successfully started 'mongodb-community@7.0' (label: homebrew.mxcl.mongodb-c
vostrov@MacBook-Pro--Ilya ~ % mongosh
Current Mongosh Log ID: 6a10122212a96c68f3a8bd27
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 7.0.34
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-22T11:21:35.723+03:00: Access control is not enabled for the database. Read and write access to data and configur
ation is unrestricted
-----

test>
```

Рис. 2: Запуск оболочки mongosh

Часть 4: Выполнение базовых методов

Выполнены команды `db.help()`, `db.help` (просмотр исходного кода JS) и `db.stats()`.

```
db.help()
db.help
db.stats()
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000 — ?directConn...
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-06-22T11:21:35.723+03:00: Access control is not enabled for the database. Read and write access to data and configur
ation is unrestricted
-----

[test> db.help()

Database Class:

getMongo                Returns the current database connection
getName                 Returns the name of the DB
getCollectionNames      Returns an array containing the names of all collections in the current datab
ase.
getCollectionInfos      Returns an array of documents with collection information, i.e. collection na
me and options, for the current database.
runCommand              Runs an arbitrary command on the database.
adminCommand            Runs an arbitrary command against the admin database.
aggregate               Runs a specified admin/diagnostic pipeline which does not require an underlyi
ng collection.
getSiblingDB            Returns another database without modifying the db variable in the shell envir
onment.
getCollection            Returns a collection or a view object that is functionally equivalent to usin
g the db.<collectionName>.
dropDatabase            Removes the current database, deleting the associated data files.
createUser              Creates a new user for the database on which the method is run. db.createUser
() returns a duplicate user error if the user already exists on the database.
updateUser              Updates the user's profile on the database on which you run the method. An up
date to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword      Updates a user's password. Run the method in the database where the user is d
efined, i.e. the database you created the user.

watch                   Opens a change stream cursor on the database
sql                     (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an exp
erimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

[test> db.help

Database Class:

getMongo                Returns the current database connection
getName                 Returns the name of the DB
getCollectionNames      Returns an array containing the names of all collections in the current datab
ase.
getCollectionInfos      Returns an array of documents with collection information, i.e. collection na
me and options, for the current database.
runCommand              Runs an arbitrary command on the database.
adminCommand            Runs an arbitrary command against the admin database.
aggregate               Runs a specified admin/diagnostic pipeline which does not require an underlyi
ng collection.
getSiblingDB            Returns another database without modifying the db variable in the shell envir
onment.
getCollection            Returns a collection or a view object that is functionally equivalent to usin
g the db.<collectionName>.
dropDatabase            Removes the current database, deleting the associated data files.
createUser              Creates a new user for the database on which the method is run. db.createUser
() returns a duplicate user error if the user already exists on the database.
updateUser              Updates the user's profile on the database on which you run the method. An up
date to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword      Updates a user's password. Run the method in the database where the user is d
efined, i.e. the database you created the user.

getLastError            Calls the getLastError command
printShardingStatus     Calls sh.status(verbose)
printSecondaryReplicatioInfo Prints secondary replicaset information
getReplicationInfo      Returns replication information
printReplicationInfo     Formats sh.getReplicationInfo
printSlaveReplicationInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk          This method is deprecated. Use db.getMongo().setReadPref() instead
watch                   Opens a change stream cursor on the database
sql                     (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an exp
erimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

[test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
[test>
```

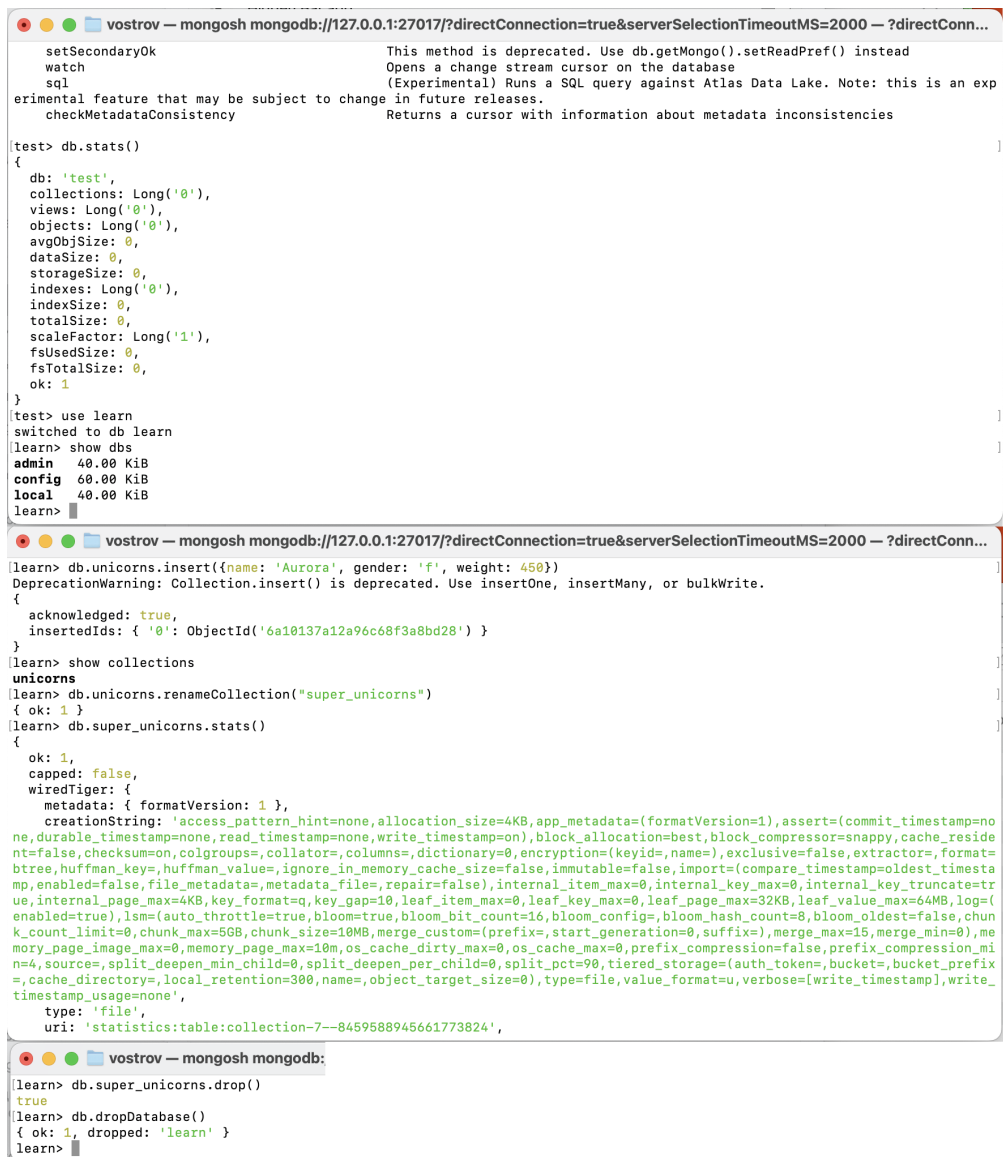
Рис. 3: Выполнение базовых методов

Часть 5: Создание БД и работа с коллекциями

Создана база данных `learn`, добавлена коллекция `unicorns`, после чего она была пере-

именована и удалена вместе с БД.

```
use learn
show dbs
db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
show collections
db.unicorns.renameCollection("super_unicorns")
db.super_unicorns.stats()
db.super_unicorns.drop()
db.dropDatabase()
```



The screenshot shows a mongosh terminal window with the following commands and output:

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000 — ?directConn...

setSecondaryOk This method is deprecated. Use db.getMongo().setReadPref() instead
watch Opens a change stream cursor on the database
sql (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}

test> use learn
switched to db learn
learn> show dbs
admin 40.00 KiB
config 60.00 KiB
local 40.00 KiB
learn>

learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10137a12a96c68f3a8bd28') }
}

learn> show collections
unicorns
learn> db.unicorns.renameCollection("super_unicorns")
{ ok: 1 }

learn> db.super_unicorns.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=on),block_allocation=best,block_compressor=snappy,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,repair=false),internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=true,bloom=true,bloom_bit_count=16,bloom_config=,bloom_hash_count=8,bloom_oldest=false,chunk_count_limit=0,chunk_max=5GB,chunk_size=10MB,merge_custom=(prefix=,start_generation=0,suffix=),merge_max=16,merge_min=0),memory_page_image_max=0,memory_page_max=10m,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,tiered_storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file,value_format=u,verbose=[write_timestamp],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-7--8459588945661773824',
  }
}

learn> db.super_unicorns.drop()
true
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn>
```

Рис. 4: Работа с БД и коллекциями

1.3 Контрольные вопросы (6.1)

1. Какую модель данных поддерживает СУБД MongoDB?

MongoDB поддерживает документо-ориентированную модель данных.

2. Какой формат данных поддерживается в MongoDB?

Для хранения данных применяется формат BSON (Binary JSON).

3. Из чего состоит база данных в MongoDB?

База данных состоит из коллекций, которые состоят из документов. То есть из пар «ключ: значение»).

4. Имеет ли БД в MongoDB жесткую структуру?

Нет, MongoDB не имеет жесткой схемы. Документы внутри одной коллекции могут иметь разный набор полей.

2 Лабораторная работа 6.2. Работа с БД в СУБД MongoDB

2.1 Цель работы

Овладеть практическими навыками работы с CRUD-операциями, вложенными объектами, агрегации, изменения данных, работы со ссылками и индексами.

2.2 Выполнение практических заданий

Часть 1: Вставка документов (Задание 2.1.1)

База данных `learn` создана заново, коллекция `unicorns` заполнена 10 документами. Дополнительный документ вставлен с помощью переменной.

```
var doc = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,
  gender: 'm', vampires: 165};
db.unicorns.insert(doc);
db.unicorns.find()
```



```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000 — ?directConn...
learn> use learn
already on db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd33') }
}
learn> db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2a') }
}
learn> db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2b') }
}
learn> db.unicorns.insert({name: 'Roooddles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2c') }
}
learn> db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2d') }
}
learn> db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2e') }
}
learn> db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd2f') }
}
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd30') }
}
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd31') }
}
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10154f12a96c68f3a8bd32') }
}
learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10156512a96c68f3a8bd34') }
}
learn> db.unicorns.find()
{
  _id: ObjectId('6a10154f12a96c68f3a8bd29'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
},
{
  _id: ObjectId('6a10154f12a96c68f3a8bd2a'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a10154f12a96c68f3a8bd2b'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
```

Рис. 5: Вставка документов

Часть 2: Выборка данных и сортировка (Задание 2.2.1)

Отработаны запросы с параметром сортировки `sort` и ограничением `limit`.

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```

    }
  ]
[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a10156512a96c68f3a8bd34'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd29'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2f'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd32'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd30'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2c'),

```

```

[learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2a'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2e'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd31'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]

```


Часть 3: Исключение полей из выборки (Задание 2.2.2)

Модифицирован запрос для вывода самцов с сокрытием полей loves и gender.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
```



The screenshot shows a terminal window titled "vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=t". The prompt is [learn>]. The command entered is `db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})`. The output is a JSON array of five objects, each representing a unicorn. Each object contains the fields `_id`, `name`, `weight`, and `vampires`. The `loves` and `gender` fields are excluded as requested.

```
[learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd29'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2b'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2c'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd2f'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd30'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd32'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd33'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  }
]
```

Рис. 7: Исключение полей из результата

Часть 4: Системная сортировка (Задание 2.2.3)

Выведен список единорогов в обратном порядке добавления с помощью `$natural`.

```
db.unicorns.find().sort({ $natural: -1 })
```

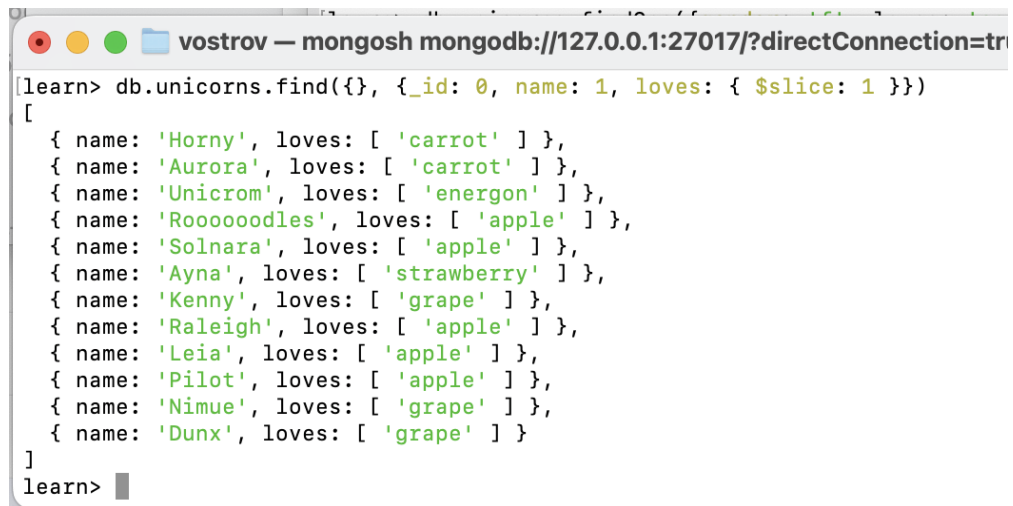
```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnectio
[learn> db.unicorns.find().sort({ $natural: -1 })
[
  {
    _id: ObjectId('6a10156512a96c68f3a8bd34'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd33'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd32'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd31'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd30'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
```

Рис. 8: Сортировка в обратном порядке

Часть 5: Работа с массивами (Задание 2.1.4)

Выведен список единорогов только с первым элементом массива предпочтений.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: { $slice: 1 }})
```



```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: { $slice: 1 }})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

Рис. 9: Использование оператора slice

Часть 6: Логические операторы (Задание 2.3.1)

Выведен список самок весом от 500 до 700 кг.

```
db.unicorns.find({gender: 'f', weight: { $gte: 500, $lte: 700 }}, {
  _id: 0 })
```

```
131 МС
vostrov — mongosh mongodb://127.0.0.1:2701
learn> db.unicorns.find({
|   gender: 'f',
|   weight: { $gte: 500, $lte: 700 }
| }, { _id: 0 })
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █
```

Рис. 10: Операторы gte и lte

Часть 7: Поиск по массивам (Задание 2.3.2)

Поиск самцов с множественными предпочтениями через оператор \$all.

```
db.unicorns.find({gender: 'm', weight: { $gte: 500 }, loves: { $all:
  ['grape', 'lemon'] }}, { _id: 0 })
```

```
vostrov — mongosh mongodb://127.0.0.1:27020
learn> db.unicorns.find({
  |   gender: 'm',
  |   weight: { $gte: 500 },
  |   loves: { $all: ['grape', 'lemon'] }
  | }, { _id: 0 })
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

Рис. 11: Поиск совпадений в массиве

Часть 8: Проверка существования поля (Задание 2.3.3)

Поиск документов без поля `vampires`.

```
db.unicorns.find({vampires: { $exists: false }})
```

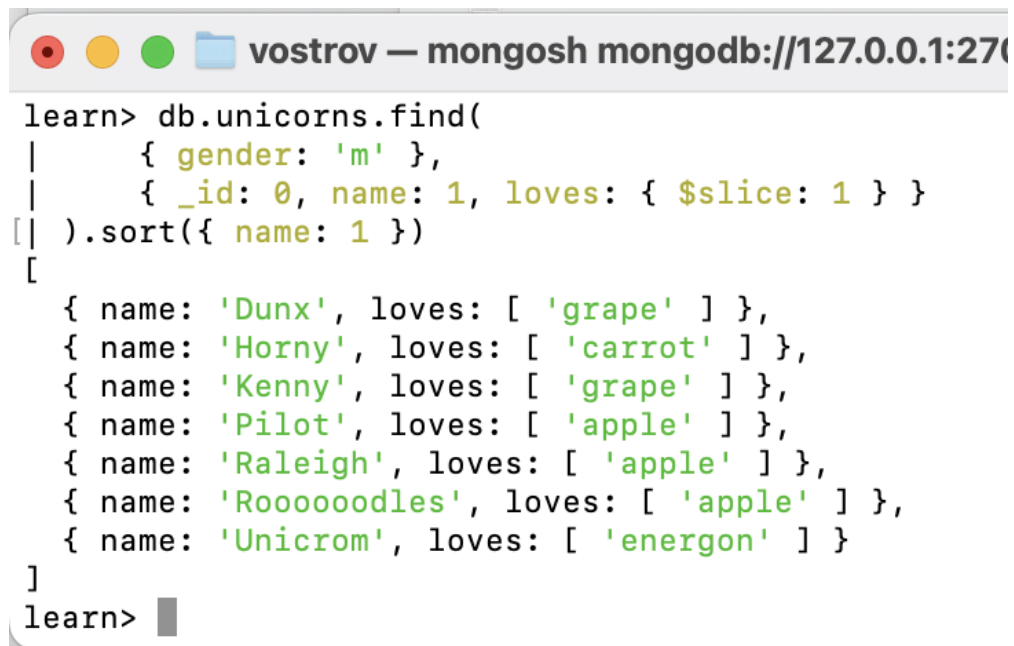
```
vostrov — mongosh mongodb://127.0.0.1:27020
learn> db.unicorns.find({
  |   vampires: { $exists: false }
  | })
[
  {
    _id: ObjectId('6a10154f12a96c68f3a8bd33'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Рис. 12: Проверка наличия поля

Часть 9: Сортировка и срез массива (Задание 2.3.4)

Комбинация `slice` и сортировки по алфавиту.

```
db.unicorns.find({ gender: 'm' }, { _id: 0, name: 1, loves: { $slice:
  1 } }).sort({ name: 1 })
```



```
learn> db.unicorns.find(
|   { gender: 'm' },
|   { _id: 0, name: 1, loves: { $slice: 1 } }
| ).sort({ name: 1 })
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

Рис. 13: Сложная выборка с сортировкой

Часть 10: Вложенные объекты (Задание 3.1.1)

Создана коллекция `towns`. Реализован поиск по вложенному объекту через точечную нотацию.

```
db.towns.find({ "mayor.party": "I" }, { _id: 0, name: 1, mayor: 1 })
db.towns.find({ "mayor.party": { $exists: false } }, { _id: 0, name:
  1, mayor: 1 })
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&

learn> db.towns.insert({
  |   name: "Punxsutawney ",
  |   populatiuon: 6200,
  |   last_sensus: ISODate("2008-01-31"),
  |   famous_for: [""],
  |   mayor: { name: "Jim Wehrle" }
  | });
[ ]

learn> db.towns.insert({
  |   name: "New York",
  |   populatiuon: 22200000,
  |   last_sensus: ISODate("2009-07-31"),
  |   famous_for: ["status of liberty", "food"],
  |   mayor: { name: "Michael Bloomberg", party: "I" }
  | });
[ ]

learn> db.towns.insert({
  |   name: "Portland",
  |   populatiuon: 528000,
  |   last_sensus: ISODate("2009-07-20"),
  |   famous_for: ["beer", "food"],
  |   mayor: { name: "Sam Adams", party: "D" }
  | });
[ ]
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1017f212a96c68f3a8bd37') }
}

learn> db.towns.find(
  |   { "mayor.party": "I" },
  |   { _id: 0, name: 1, mayor: 1 }
  | )
[ ]
[ { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } } ]

learn> db.towns.find(
  |   { "mayor.party": { $exists: false } },
  |   { _id: 0, name: 1, mayor: 1 }
  | )
[ ]
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]

learn>
```

Рис. 14: Работа со вложенными документами

Часть 11: Работа с курсорами (Задание 3.1.2)

Перебор полученных результатов через курсор и метод `forEach`.

```
var cursor = db.unicorns.find({gender: 'm'}); null;
cursor.sort({name: 1}).limit(2); null;
cursor.forEach(function(obj) { print(obj.name); })
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directC

learn> var cursor = db.unicorns.find({gender: 'm'}); null;
| cursor.sort({name: 1}).limit(2); null;
| cursor.forEach(function(obj) {
|   print(obj.name);
| })
Dunx
Horny

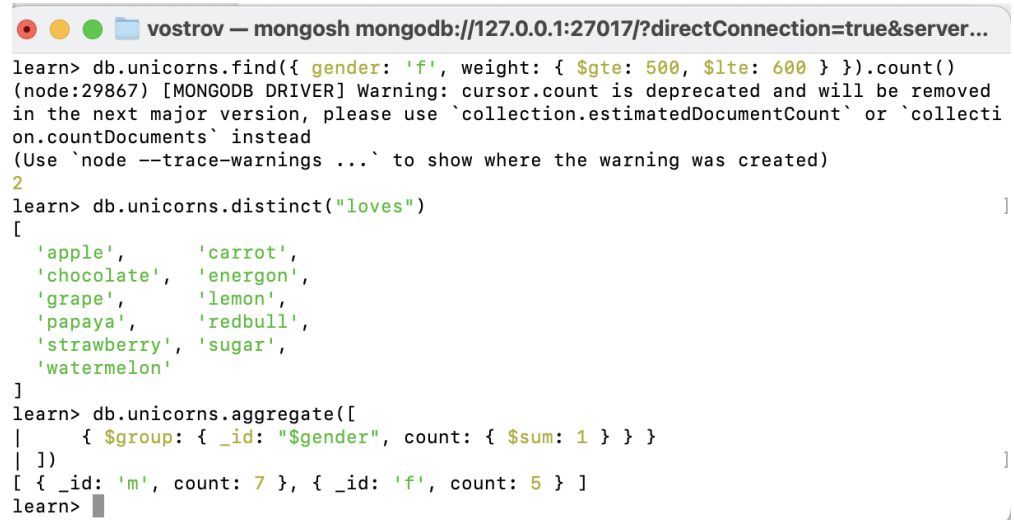
learn>
```

Рис. 15: Перебор данных курсором

Часть 12: Агрегированные запросы (Задания 3.2.1 - 3.2.3)

Подсчет, выборка уникальных значений и группировка (аналог GROUP BY).

```
db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 600 } }).count()
db.unicorns.distinct("loves")
db.unicorns.aggregate([ { $group: { _id: "$gender", count: { $sum: 1 } } } ])
```



The screenshot shows a terminal window titled "vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&server...". It displays the following commands and their outputs:

```
learn> db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 600 } }).count()
(node:29867) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed
in the next major version, please use `collection.estimatedDocumentCount` or `collecti
on.countDocuments` instead
(Use `node --trace-warnings ...` to show where the warning was created)
2
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> db.unicorns.aggregate([
  |   { $group: { _id: "$gender", count: { $sum: 1 } } }
  | ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn>
```

Рис. 16: Агрегация данных

Часть 13: Редактирование с insertOne и updateOne (Задания 3.3.1 - 3.3.2)

```
db.unicorns.insertOne({ name: 'Barney', loves: ['grape'], weight: 340,
  gender: 'm' })
db.unicorns.updateOne({ name: 'Ayna' }, { $set: { weight: 800,
  vampires: 51 } })
```



```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTi...
[learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
[learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a11743e12a96c68f3a8bd38')
}
[learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a11743e12a96c68f3a8bd38'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
[learn> db.unicorns.updateOne(
  { name: 'Ayna' },
  { $set: { weight: 800, vampires: 51 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> ]
```

Рис. 17: Вставка и редактирование документов

Часть 14: Точечное изменение данных (Задания 3.3.3 - 3.3.5)

Использование инкремента (\$inc) и удаления полей (\$unset).

```
db.unicorns.updateOne({ name: 'Raleigh' }, { $set: { loves: ['redbull', ''] } })
db.unicorns.updateMany({ gender: 'm' }, { $inc: { vampires: 5 } })
db.towns.updateOne({ name: 'Portland' }, { $unset: { "mayor.party": 1 } })
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTi...
[learn> db.unicorns.updateOne(
  { name: 'Raleigh' },
  { $set: { loves: ['redbull'] } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.updateMany(
  { gender: 'm' },
  { $inc: { vampires: 5 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
[learn> db.towns.updateOne(
  { name: 'Portland' },
  { $unset: { "mayor.party": 1 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> ]
```

Рис. 18: Изменение числовых данных и структуры

Часть 15: Работа с массивами при обновлении (Задания 3.3.6 - 3.3.7)

Добавление элементов в массив через \$push и \$each.

```
db.unicorns.updateOne({ name: 'Pilot' }, { $push: { loves: 'chocolate' } })
db.unicorns.updateOne({ name: 'Aurora' }, { $push: { loves: { $each: ['sugar', 'lemon'] } } })
```



```
vostrov — mongosh mongodb://127.0.0.1:27017/?di...
learn> db.unicorns.updateOne(
|   { name: 'Pilot' },
|   { $push: { loves: 'chocolate' } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
|   { name: 'Aurora' },
|   { $push: { loves: { $each: ['sugar', 'lemon'] } } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> █
```

Рис. 19: Добавление данных в массивы

Часть 16: Удаление данных (Задание 3.4.1)

Очистка коллекции towns с помощью deleteMany.

```
db.towns.deleteMany({ "mayor.party": { $exists: false } })
db.towns.deleteMany({})
show collections
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection...
learn> db.towns.deleteMany({ "mayor.party": { $exists: false } })
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a1017f212a96c68f3a8bd36'),
    name: 'New York',
    populatioun: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
learn> show collections
towns
unicorns
learn> █
```

Рис. 20: Удаление документов

Часть 17: Работа со ссылками DBRef (Задание 4.1.1)

Создана коллекция зон (zones), добавлена ссылка DBRef в документ единорога.

```
db.zones.insertOne({_id: "forest", fullName: "Enchanted Forest",
description: "
"})
db.unicorns.updateOne({ name: 'Aurora' }, { $set: { homeZone: DBRef("
zones", "forest") } })
db.unicorns.findOne({ name: 'Aurora' })
```

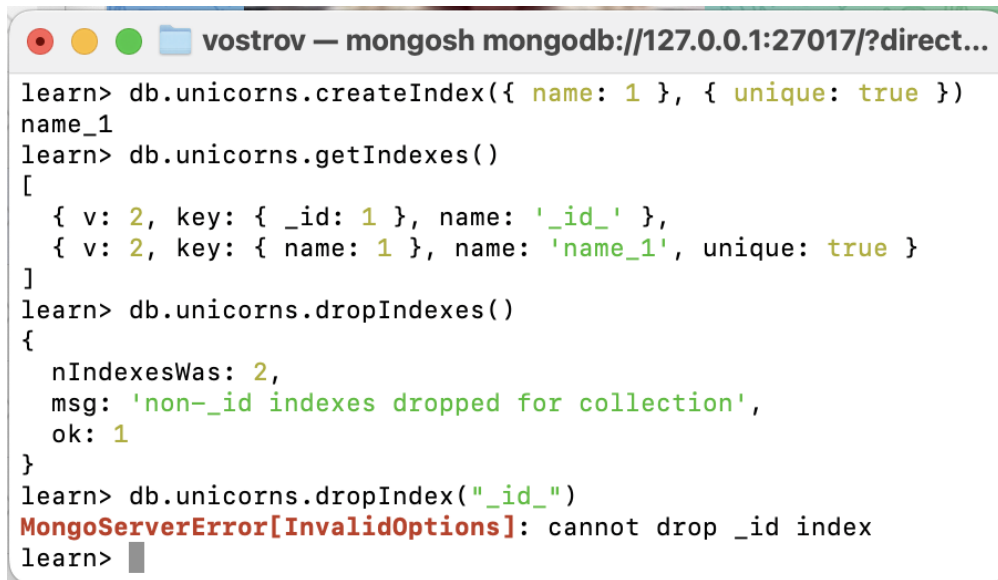
```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection...
learn> db.zones.insertOne({
  _id: "forest",
  fullName: "Enchanted Forest",
  description: "Темный магический лес с древними деревьями"
})
{ acknowledged: true, insertedId: 'forest' }
learn> db.unicorns.updateOne(
  { name: 'Aurora' },
  { $set: { homeZone: DBRef("zones", "forest") } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.findOne({ name: 'Aurora' })
{
  _id: ObjectId('6a10154f12a96c68f3a8bd2a'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
  weight: 450,
  gender: 'f',
  vampires: 43,
  homeZone: DBRef('zones', 'forest')
}
learn> █
```

Рис. 21: Организация связей DBRef

Часть 18: Создание и удаление Индексов (Задания 4.2.1 и 4.3.1)

Создан уникальный индекс для ключа `name`. Произведена попытка удаления системного индекса `_id_`.

```
db.unicorns.createIndex({ name: 1 }, { unique: true })
db.unicorns.getIndexes()
db.unicorns.dropIndexes()
db.unicorns.dropIndex("_id_")
```



```
vostrov — mongosh mongodb://127.0.0.1:27017/?direct...
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Рис. 22: Управление индексами

Часть 19: План запроса и производительность (Задание 4.4.1)

Сгенерировано 100 000 документов. Замерена скорость выборки без индекса и с индексом с помощью `explain("executionStats")`.

```
var docs = []; for(var i=0; i<100000; i++){ docs.push({value: i}) };
db.numbers.insertMany(docs);

db.numbers.find().sort({value: -1}).limit(4).explain("executionStats"
)

db.numbers.createIndex({ value: 1 })
db.numbers.find().sort({value: -1}).limit(4).explain("executionStats"
)
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?direct...
learn> var docs = [];
| for(var i = 0; i < 100000; i++) {
|   docs.push({value: i});
| }
[| db.numbers.insertMany(docs);
| ]
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a11773b12a96c68f3a8bd39'),
    '1': ObjectId('6a11773b12a96c68f3a8bd3a'),
    '2': ObjectId('6a11773b12a96c68f3a8bd3b'),
    '3': ObjectId('6a11773b12a96c68f3a8bd3c'),
    '4': ObjectId('6a11773b12a96c68f3a8bd3d'),
    '5': ObjectId('6a11773b12a96c68f3a8bd3e'),
    '6': ObjectId('6a11773b12a96c68f3a8bd3f'),
    '7': ObjectId('6a11773b12a96c68f3a8bd40'),
    '8': ObjectId('6a11773b12a96c68f3a8bd41'),
    '9': ObjectId('6a11773b12a96c68f3a8bd42'),
    '10': ObjectId('6a11773b12a96c68f3a8bd43'),
    '11': ObjectId('6a11773b12a96c68f3a8bd44'),
    '12': ObjectId('6a11773b12a96c68f3a8bd45'),
  }
}
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serve...
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {},
    queryHash: '0DB68434',
    planCacheKey: '0DB68434',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 71,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 1,
      works: 100006,
      advanced: 4,
      needTime: 100001,
      needYield: 0,
    }
  }
}
```

```
vostrov — mongosh mongodb://127.0.0.1:27017/?directConnection=true&serve...
[learn> db.numbers.createIndex({ value: 1 })
value_1
[learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {},
    queryHash: '0DB68434',
    planCacheKey: '0DB68434',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
        }
      }
    }
  }
}
```

Вывод к п. 4.4.1: Запрос с индексом работает в десятки раз эффективнее. Без индекса СУБД выполняет полное сканирование коллекции, проверяя каждый из 100 000 документов. При наличии индекса СУБД сканирует непосредственно индексное дерево, из-за чего количество просмотренных документов совпадает с `limit`, а время выполнения падает до ≈ 0 мс.

2.3 Контрольные вопросы (6.2)

1. Как используется оператор точка?

Оператор точка используется для доступа к полям вложенных документов. Например, "mayor.party".

2. Как можно использовать курсор?

Курсор хранит указатель на результат выборки. Используя методы JS (`hasNext`, `next`, `forEach`), можно перебирать документы в цикле, не загружая их все разом в оперативную память.

3. Какие возможности агрегирования данных существуют в MongoDB?

Методы `count()`, `distinct()`, а также метод `aggregate()`, принимающий конвейер операций из `$group`, `$sum`, `$match`, что заменяет GROUP BY из SQL.

4. Какая из функций `save` или `update` более детально позволит настроить редактирование?

В современных версиях MongoDB используются `updateOne` и `updateMany`. Они позволяют точно изменять документы с помощью модификаторов: `$set`, `$inc`, `$push`, не перезаписывая документ целиком.

5. Как происходит удаление документов из коллекции по умолчанию?

Функция `deleteMany` удаляет все документы, попадающие под условие выборки. Если передать пустой объект `{}`, будут удалены все документы коллекции.

6. Назовите способы связывания коллекций в MongoDB.

Связывание может быть ручным, то есть сохранение `_id` в строковое поле, и автоматическим с помощью типа данных `DBRef`, который явно хранит имя коллекции и `_id` связанного документа.

7. Сколько индексов можно установить на одну коллекцию в БД MongoDB?

MongoDB позволяет создать до 64 индексов на одну коллекцию.

8. Как получить информацию о всех индексах базы данных MongoDB?

Информация извлекается методом `getIndexes()`.

2.4 Вывод

В ходе выполнения лабораторных работ были успешно освоены практические навыки работы с СУБД MongoDB. Изучены базовые CRUD-операции, сложные выборки, а также работа с курсорами и агрегация данных. На практике была доказана высочайшая производительность поиска по коллекции при использовании индексов по сравнению с прямым сканированием коллекции.